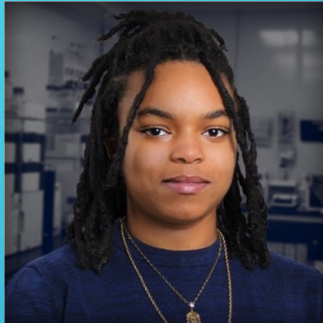




Phishing Detection



Nathan Poch
Masters in Cyber Security



Maurisa Dacosta
Masters in Computer
Science



Rodrigo Arguello
Masters in Computer
Science



Common Phishing Types



Phishing

Uses **mass emails** to trick **individuals and groups** into revealing sensitive information.



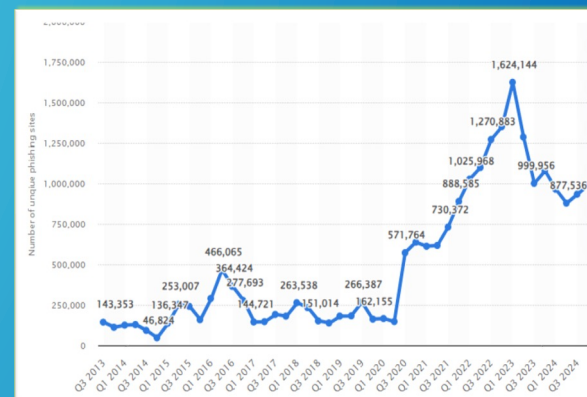
Spear Phishing

Uses **personalized emails** to trick **individuals** into revealing information.



Whaling

Uses **personalized emails** to trick **high-value targets** into revealing information.



The number of detected phishing sites across the world in each quarter from the 3rd quarter of 2013 to the 4th quarter of 2024 ([Statista, 2025](#)).



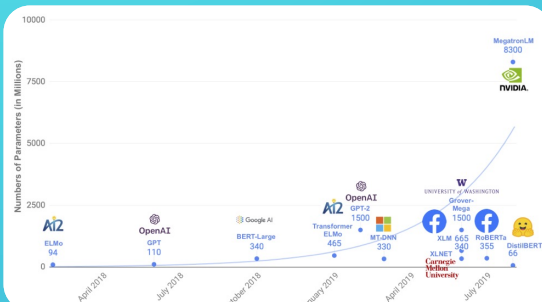
Feature Extraction Using DistilBERT

$$q_i = \text{Softmax} \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

$$L_{ce} = \sum_i t_i * \log(s_i)$$

$$L_{mlm} = - \sum_i y_i \log(p_i)$$

$$L_{cos} = 1 - \cos(h_t, h_s) = 1 - \frac{h_t \cdot h_s}{||h_t|| \times ||h_s||}$$



- ❖ 40% smaller, 60% faster than BERT
- ❖ 97% of BERT's performance
- ❖ Extracts semantic features from phishing emails/URLs

Table 1: DistilBERT retains 97% of BERT performance. Comparison on the dev sets of the GLUE benchmark. ELMo results as reported by the authors. BERT and DistilBERT results are the medians of 5 runs with different seeds.

Model	Score	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	WNLI
ELMo	68.7	44.1	68.6	76.6	71.1	86.2	53.4	91.5	70.4	56.3
BERT-base	79.5	56.3	86.7	88.6	91.8	89.6	69.3	92.7	89.0	53.5
DistilBERT	77.0	51.3	82.2	87.5	89.2	88.5	59.9	91.3	86.9	56.3

Table 2: DistilBERT yields to comparable performance on downstream tasks. Comparison on downstream tasks: IMDb (test accuracy) and SQuAD 1.1 (EM/F1 on dev set). D: with a second step of distillation during fine-tuning.

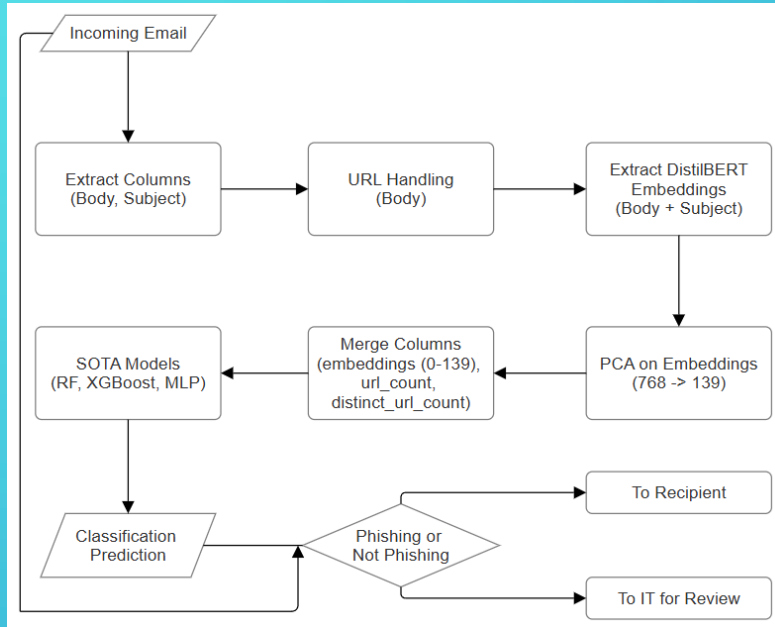
Model	IMDb (acc.)	SQuAD (EM/F1)
BERT-base	93.46	81.2/88.5
DistilBERT	92.82	77.7/85.8
DistilBERT (D)	-	79.1/86.9

Table 3: DistilBERT is significantly smaller while being constantly faster. Inference time of a full pass of GLUE task STS-B (sentiment analysis) on CPU with a batch size of 1.

Model	# param. (Millions)	Inf. time (seconds)
ELMo	180	895
BERT-base	110	668
DistilBERT	66	410

DistilBERT (D)	-	38.1/88.9
DistilBERT	85.85	13.2/82.8
BERT-base	81.90	81.2/88.5
ELMo	180	895

Architecture Overview



The Data (Pre-Cleaning)

```
Nazario_df.iloc[1]
```

✓ 0.0s

```
sender      Drop          cPanel <service@cpanel.com>  
receiver    Drop          jose@monkey.org  
date        Drop          Fri, 30 Oct 2015 00:00:48 -0500  
subject     Verify Your Account  
body        Business with \t\t\t\t\t\t\t\tcPanel & WHM \t...  
urls        Replace (url_count, distinct_url_count)          1  
label       1  
Name: 1, dtype: object
```

- 1) Combined (subject + " " + body).
- 2) Passed to DistilBERT for embeddings
- 3) Embeddings passed through PCA (95% entropy)

} URL extraction and replacement
(all URLs replaces with <url>)

```
Ling_df.iloc[1746]
```

✓ 0.0s

```
subject      dick armey 's slip and correction
body         i was curious if anyone was actually intereste...
label        0
Name: 1746, dtype: object
```

Feature Extraction with DistilBERT and PCA

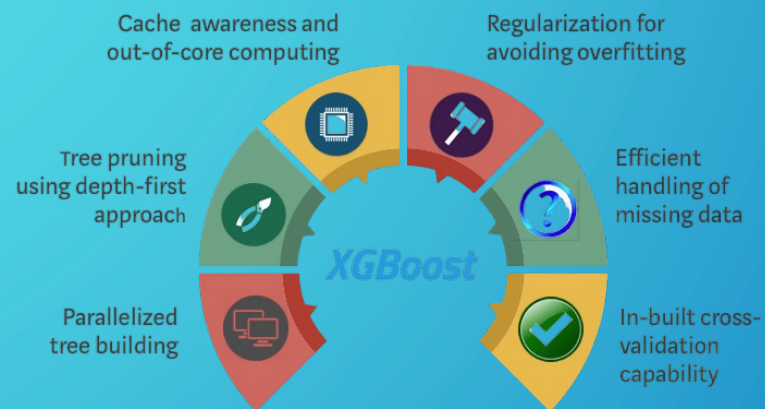
```
# Combine text columns, drop unnecessary columns
df_in['text'] = df_in['subject'] + " " + df_in['body']
df_pre_pca = df_in.drop(columns=['body', 'subject', 'Unnamed: 0'])
```

```
if pca is None:
    # Define and use PCA
    pca = PCA(n_components=target_variance)
    scaler = StandardScaler()
    scaler.fit(embedding_df)
    embedding_scaled_df = scaler.transform(embedding_df)
    pca.fit(embedding_scaled_df)
    embedding_reduced_df = pca.transform(embedding_scaled_df)
    embedding_reduced_df = pd.DataFrame(embedding_reduced_df, index=df.index)
    embedding_reduced_df.columns = [f'pca_{i}' for i in embedding_reduced_df.columns]

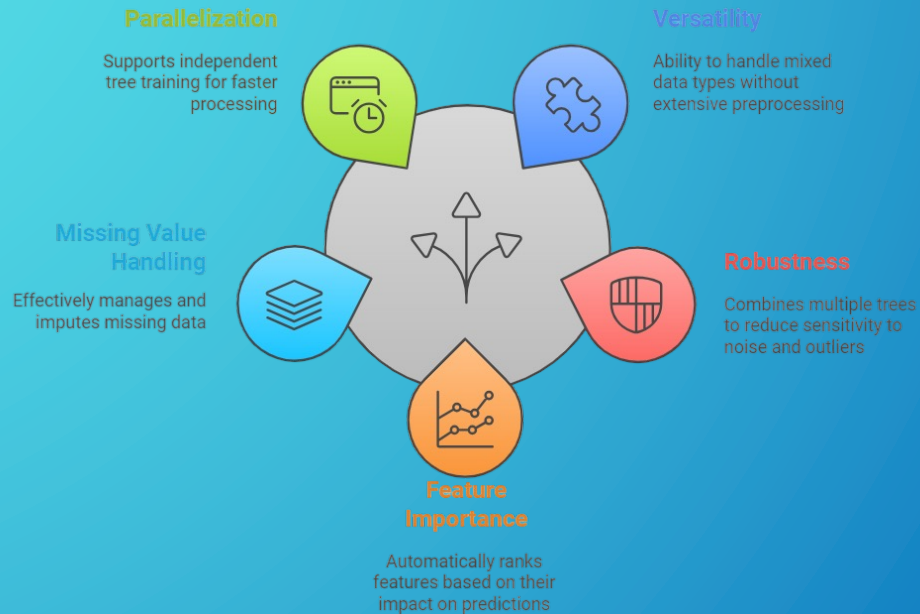
else:
    if scaler is None:
        raise ValueError("Please provide a scaler.")
    embedding_scaled_df = scaler.transform(embedding_df)
    embedding_reduced_df = pca.transform(embedding_scaled_df)
    embedding_reduced_df = pd.DataFrame(embedding_reduced_df, index=df.index)
    embedding_reduced_df.columns = [f'pca_{i}' for i in embedding_reduced_df.columns]
    # Use provided PCA
```

```
self.tokenizer = AutoTokenizer.from_pretrained("distilbert-base-cased")
self.model = AutoModel.from_pretrained("distilbert-base-cased")
```

Why XG Boost ?



Why Random Forest ?



Random Forest & XGBoost Two-Stage Tuning Strategy

Tuning Stage	Random Forest	XG Boost
1. Randomized Search Parameters (3-fold Cross Validation)	n_estimators: (300,600) max_depth: (5,30) min_samples_split: (5,15) min_samples_leaf: (2,15)	n_estimators: (100,300) max_depth: (3,8) learning_rate: (0.01,0.2) subsample: (0.6,0.4)
2. Grid Search Parameters (5-fold Cross Validation)	n_estimators: [400,420,442,462,482] max_depth: [17,19,21,23] min_samples_split: [8,10,12] min_samples_leaf: [3,4,5,6]	n_estimators: [225,245,265] max_depth: [5,6,7] learning_rate: [0.12, 0.135, 0.145, 0.15] subsample: [0.8, 0.845, 0.9]

Random Forest Model

Stage 1: Tuning Hyperparameters with Randomized Search

```
#Random Forest Model with Parameter tuning using RandomizedSearchCV
rf_params = {
    'n_estimators': randint(300, 600),
    'max_depth': randint(5, 30),
    'min_samples_split': randint(5, 15),
    'min_samples_leaf': randint(2, 15),
}

rf_search = RandomizedSearchCV(RandomForestClassifier(random_state=42), rf_params, n_iter=10, cv=3, n_jobs=-1, verbose=1)
```

Stage 2: Tuning with GridSearch Based on Performance of Randomized Search

```
#Random Forest Model with Parameter tuning using GridSearchCV
rf_params = {
    'n_estimators': [400,420,442,462,482],
    'max_depth': [17,19,21,23],
    'min_samples_split': [8,10,12],
    'min_samples_leaf': [3,4,5,6],
}

rf_search = GridSearchCV(RandomForestClassifier(random_state=42), rf_params, cv=5, n_jobs=-1, verbose=1)
```

XG Boost Model

Stage 1: Tuning Hyperparameters with Randomized Search

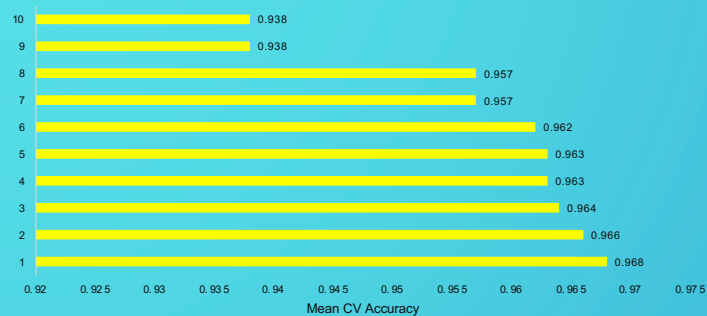
```
#XG Boost Model with Parameter tuning using RandomizedSearchCV
xgb_params = {
    'n_estimators': randint(100, 300),
    'max_depth': randint(3, 8),
    'learning_rate': uniform(0.01, 0.2),
    'subsample': uniform(0.6, 0.4),
}
xgb_search = RandomizedSearchCV(XGBClassifier(random_state=42, eval_metric='logloss'), xgb_params, n_iter=10, cv=3, n_jobs=-1, verbose=1)
```

Stage 2: Tuning with GridSearch Based on Performance of Randomized Search

```
#XG Boost Model with Parameter tuning using GridSearchCV
xgb_params = {
    'n_estimators': [225,245,265],
    'max_depth': [5,6,7],
    'learning_rate': [0.12,0.135,0.145,0.15],
    'subsample': [0.8,0.845,0.9],
}
xgb_search = GridSearchCV(XGBClassifier(random_state=42, eval_metric='logloss'), xgb_params, cv=5, n_jobs=-1, verbose=1)
xgb_search.fit(X_train, y_train)
```

Randomized Search Top Combinations

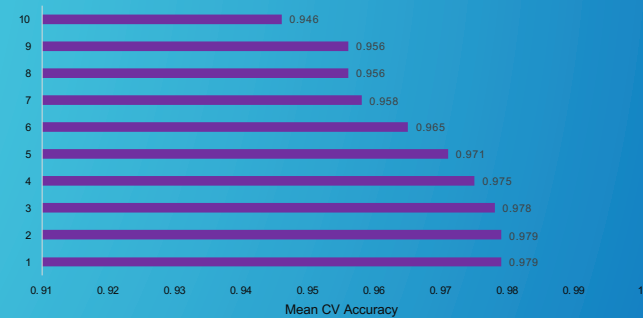
Top Random Forest Parameter Combinations



Legend Key

Parameters	max_depth	min_sample s_leaf	min_samples_ split	n_estimators
1	19	4	10	442
2	20	6	6	499
3	19	10	10	412
4	24	11	6	488
5	20	13	12	464
6	27	12	5	329
7	12	14	13	583
8	11	9	12	326
9	7	10	8	592
10	7	5	8	408

Top XGBoost Parameter Combinations



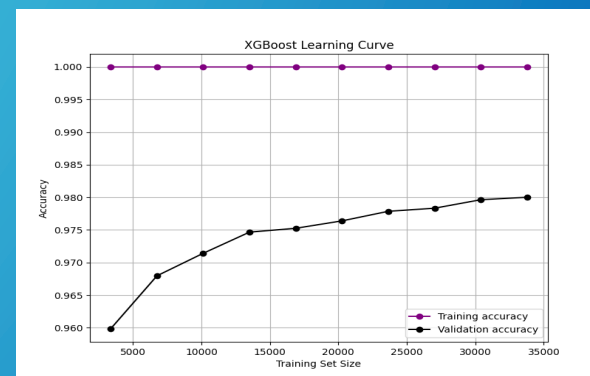
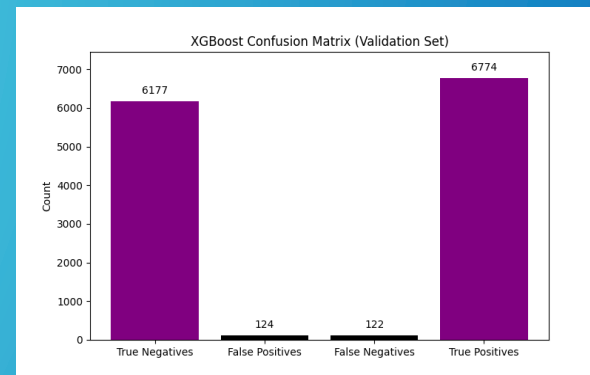
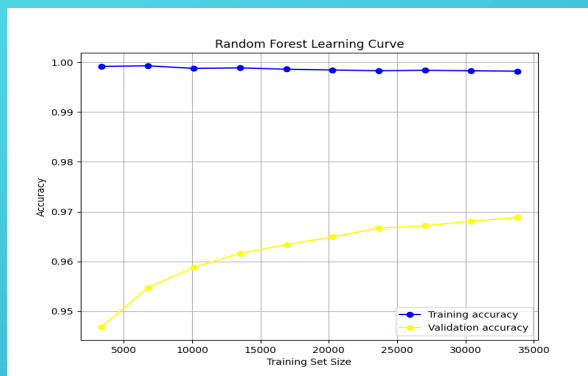
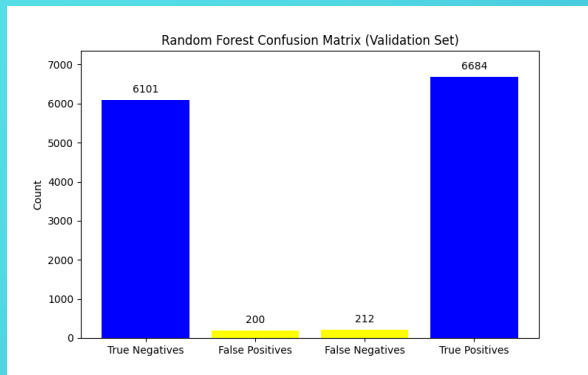
Legend Key

Parameters	learning_rate	max_depth	subsample	n_estimators
1	0.145	7	0.845	245
2	0.150	6	0.948	223
3	0.101	7	0.973	237
4	0.132	5	0.773	156
5	0.084	5	0.708	156
6	0.073	3	0.652	278
7	0.122	3	0.907	120
8	0.112	3	0.965	121
9	0.011	7	0.777	239
10	0.043	3	0.871	193

Randomized Search Metrics

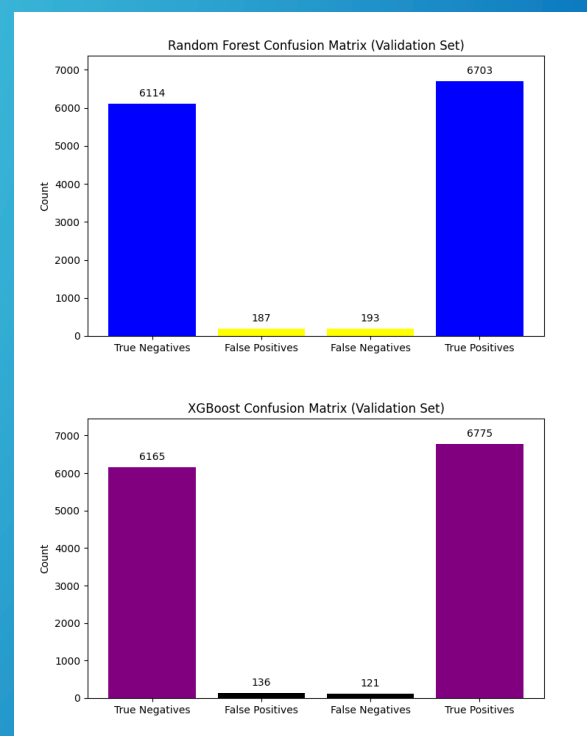
Metric	Random Forest	XG Boost
Top Params	n_estimators: 442 max_depth: 19 min_samples_split: 10 min_samples_leaf: 4	n_estimators: 245 max_depth: 7 learning_rate: 0.145 subsample: 0.845
Train / Val Accuracy	99.83% / 96.88%	100.00% / 98.14%
Precision	Class 0: 0.97 Class 1: 0.97	Class 0: 0.98 Class 1: 0.98
Recall	Class 0: 0.97 Class 1: 0.97	Class 0: 0.98 Class 1: 0.98
F1-Score	Macro: 0.97 Weighted: 0.97	Macro: 0.98 Weighted: 0.98
Overall Accuracy	96.88%	98.14%

Randomized Search Insights



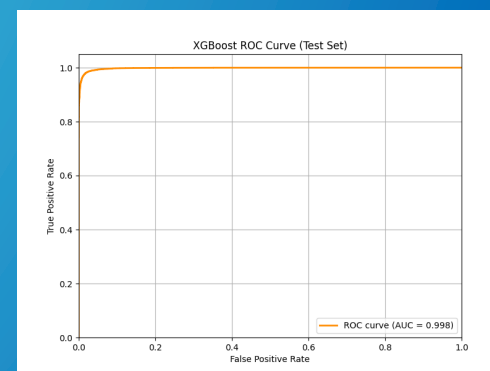
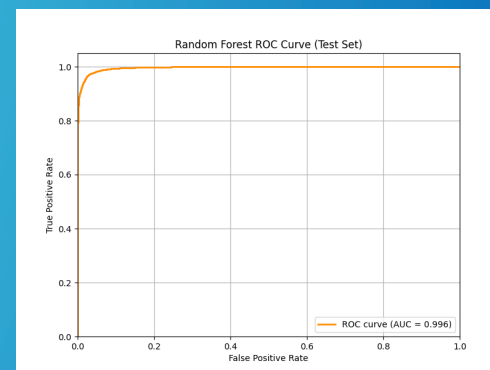
Grid Search Metrics & Insights

Metric	Random Forest	XG Boost
Top Params	n_estimators: 482 max_depth: 23 min_samples_split: 8 min_samples_leaf: 3	n_estimators: 265 max_depth: 7 learning_rate: 0.15 subsample: 0.845
Train / Val Accuracy	99.94% / 97.12%	100.00% / 98.05%
Precision	Class 0: 0.97 Class 1: 0.97	Class 0: 0.98 Class 1: 0.98
Recall	Class 0: 0.97 Class 1: 0.97	Class 0: 0.98 Class 1: 0.98
F1-Score	Macro: 0.97 Weighted: 0.97	Macro: 0.98 Weighted: 0.98
Overall Accuracy	97.12%	98.05%



Performance of Tuned Models on Test Data

Metric	Random Forest	XGBoost
True Positives (TP)	8245	8353
True Negatives (TN)	7753	7823
False Positives (FP)	223	163
False Negatives (FN)	266	153
Precision	Class 0: 0.967 Class 1: 0.973	Class 0: 0.981 Class 1: 0.981
Recall	Class 0: 0.971 Class 1: 0.969	Class 0: 0.980 Class 1: 0.982
F1-Score	Macro: 0.970 Weighted: 0.970	Macro: 0.981 Weighted: 0.981
Overall Accuracy	96.98%	98.08%



Why MLP ?

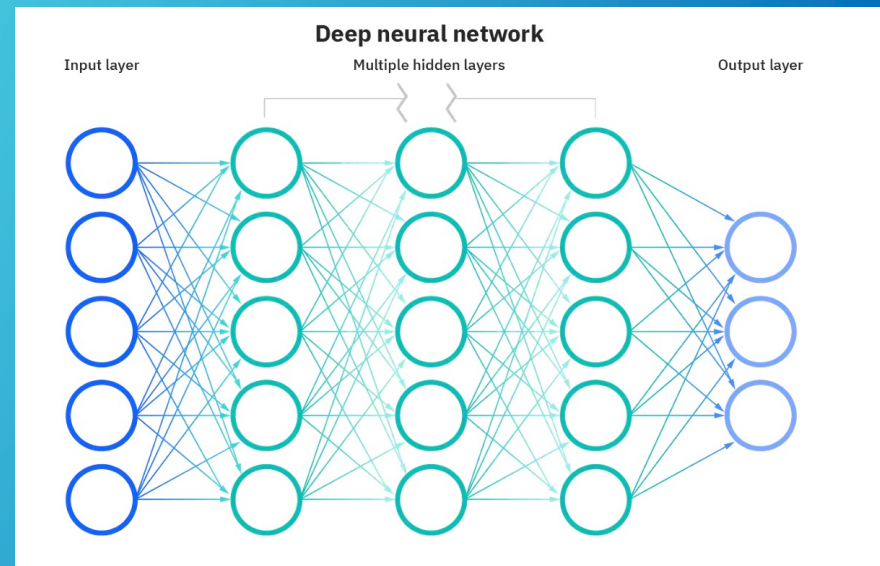
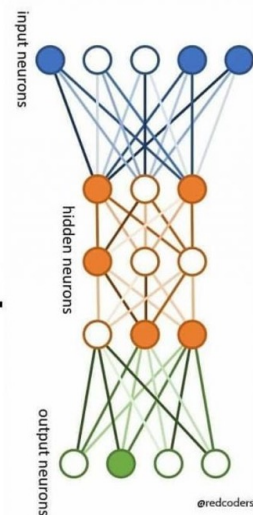


Multi-Layer-Perceptron Model (MLP)

THIS IS A NEURAL NETWORK.

**IT MAKES MISTAKES.
IT LEARNS FROM THEM.**

BE LIKE A NEURAL NETWORK.



Multi-Layer-Perceptron Model - Metrics

Precision: How many predicted phishing emails were actually phishing?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: How many actual phishing emails were correctly detected?

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The pipeline was implemented using the following components:

- **Libraries:**

- `TensorFlow / Keras` : For defining, training, and saving the MLP model.
- `Scikit-learn` : For evaluation metrics such as accuracy, precision, recall, F1, FPR, FNR.
- `Pandas, Numpy` : For data loading and manipulation.
- `Matplotlib` : For plotting learning curves.

- **Key Classes/Functions:**

- `Sequential` : Model constructor for stacking dense layers.
- `Dense` : For MLP layers with ReLU and sigmoid activations.
- `EarlyStopping`, `TimeStopping` : Training callbacks to stop based on criteria.
- `eval_classification` : Custom evaluation to compute all relevant metrics and visualize confusion matrix.
- `plot_history` : Helper to visualize training/validation curves.

- **Hyperparameter Search Space:**

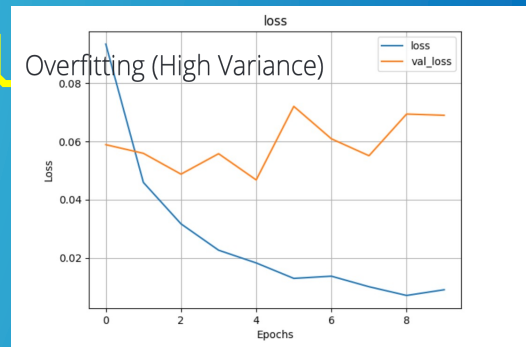
- Layer configs: `[[64], [128, 64], [256, 128, 64]]`
- Epochs: `[50, 100]`
- Min delta: `[0.0, 0.001]`
- Patience: `[1, 3, 5]`

Multi-Layer-Perceptron Model Preliminary Results

🏆 Best Model Configuration:

layers: [128, 64]
epochs: 100
min_delta: 0.0
patience: 5
val_f1_score: 0.9877438537965044

📁 Best model saved as 'best_mlp_model.h5'
Results saved to mlp_model_results.csv



📊 Top Validation Results by F1 Score:

	Accuracy	Precision	Recall	F1 Score	FPR	FNR
val [128, 64]-100-p5-d0.0	0.987194	0.987959	0.987529	0.987744	0.013173	0.012471
val [128, 64]-100-p3-d0.001	0.987043	0.987955	0.987239	0.987597	0.013173	0.012761
val [256, 128, 64]-100-p5-d0.001	0.986664	0.986252	0.988254	0.987252	0.015077	0.011746
val [256, 128, 64]-50-p3-d0.001	0.986436	0.987233	0.986804	0.987019	0.013966	0.013196
val [128, 64]-100-p5-d0.001	0.986285	0.985258	0.988544	0.986898	0.016188	0.011456
val [256, 128, 64]-50-p3-d0.0	0.985982	0.987081	0.986079	0.986580	0.014125	0.013921
val [256, 128, 64]-100-p3-d0.0	0.985830	0.984404	0.988544	0.986470	0.017140	0.011456
val [64]-100-p3-d0.001	0.985830	0.985948	0.986949	0.986448	0.015394	0.013051
val [256, 128, 64]-100-p5-d0.0	0.985527	0.986504	0.985789	0.986146	0.014760	0.014211
val [256, 128, 64]-50-p5-d0.0	0.985300	0.982853	0.989124	0.985979	0.018886	0.010876

Attacking High Variance:

- Early Stopping ☒
- Dropout Layers ☒
- L1 and L2 Regularization ☒
- Decrease model complexity ☒

Multi-Layer-Perceptron Model

Fin

🏆 Best Model Configuration:|
layers: [64]
epochs: 100
min_delta: 0.001
patience: 7
dropout_rate: 0.3
val_f1_score: 0.9642547033285094

=== Champion Model on Validation Dataset ===

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

No Phishing Email	0.96	0.96	0.96	6301
Phishing Email	0.96	0.97	0.96	6896
accuracy			0.96	13197
macro avg	0.96	0.96	0.96	13197
weighted avg	0.96	0.96	0.96	13197

	Accuracy	Precision	Recall	F1 Score	FPR	FNR
Best Model on Validation	0.962567	0.962305	0.966212	0.964255	0.041422	0.033788

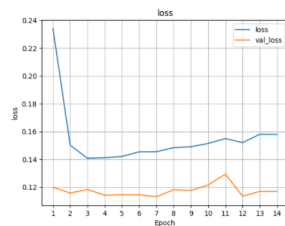


Figure 22: Champion MLP Model Loss History showing some variance, but performing well over time

```
# --- Hyperparameter Space ---  
layer_configs = [[64], [64, 64], [64, 128, 64], [256, 128, 128, 256]]  
epochs_list = [50, 100, 200, 500]  
min_deltas = [0.0, 0.001]  
patiences = [3, 5, 7, 9]  
dropout_rates = [0.3, 0.5]
```

Lessons Learned,

Maurisa Dacosta Masters in Computer Science

Coming from a chemistry background, this has been my first in depth experience with AI and machine learning. It has been exciting teaching myself how to build, train, and tune models as well as doing in depth research on what each hyperparameter means and how it influences a models performance. Also learning performance metrics and the significance behind them helped me broaden my technical skills. I developed a stronger understanding of a machine learning workflow and practiced executing these technical concepts in visualizations for presentations and documentation.

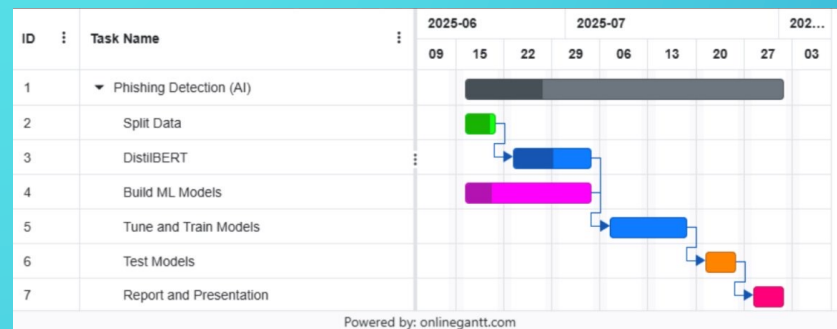
Nathan Poch Masters in Cyber Security

My background in mathematics, cybersecurity, and machine learning has helped me focus in on some of the small details during our data cleaning phase. While I have learned about DistilBERT and how to use it to turn textual data into model-ready tokens, the biggest lessons to learn came from the URL extraction portion of our work. The Top Level Domain based approach offered by URLExtract provided an output on the example email I was analysing that differed from the regular-expression based method in CyberChef. This led me to build a class for extracting URLs that used both URLExtract and RegEx, giving me the best of both worlds, and serving as a reminder to always test the solutions provided by others. Also, while fixing a minor bug in one of the classes I built for this phase, I realized how convenient it was to fix bugs in a class one time as opposed to fixing the same bug in several locations in the code, providing further incentive to take the hard route the first time. Many of my math professors told me, "The great thing about mathematicians is that they are lazy." This was in reference to making and proving theorems and propositions so that they would not need to solve the same problem over and over again. This bug fixing experience showed me the connection between theorem proving in math and class building in computer science, which I think was a valuable lesson in software engineering.

Rodrigo Arguello Masters in Computer Science

Applying these AI and machine learning lessons can enhance aerospace manufacturing tests by improving anomaly detection, automating data analysis, and optimizing test processes. Understanding model tuning, performance metrics, and modular code design enables the development of smarter systems for identifying early faults, reducing test time, and increasing product reliability, ultimately ensuring higher safety and quality standards across complex aerospace production environments.

Conclusions & Future Work



"By the end of the semester, our group produced a working phishing-detection pipeline that, line-by-line and in every detail, matches the milestones we established in our initial project plan. We (i) integrated six unrelated e-mail corpora, (ii) constructed effective URL-based features, (iii) produced DistilBERT embeddings, (iv) dimensionally reduced them with PCA, and (v) trained and tuned three state-of-the-state classifiers—Random Forest, XGBoost and a lightweight MLP—any one of which now reaches $\geq 96\%$ accuracy and an $AUC \approx 0.99$ on a previously unseen test set. All of our code compiles and executes end-to-end in a reproducible environment. We can show that the problem we selected was neither too grand nor too small: it fell within scope. Still, it yielded immediate applicability in a real-time e-mail-filtering scenario. Each task in our Gantt chart—from data ingestion (Week 2) to final documentation (Week 10)—was either accomplished ahead of schedule or exactly on time. Our Team 5 project meets the goals we established at its initiation." Team 5

References

Used in Presentation:

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108.

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019, June). Bert: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers) (pp. 4171-4186).

Owen, A., & White, E. (2024). Evaluating the Effectiveness of AI-Based Phishing Detection in IoT Communication Protocols.

Morde, V. (2019, June 23). *XGBoost algorithm: Long may she reign!* Medium. <https://medium.com/data-science/https-medium-com-vishalmorde-xgboost-algorithm-long-she-may-rein-edd9f99be63d>

SitePoint. (2025). Random Forest algorithm in machine learning. <https://www.sitepoint.com/random-forest-algorithm-in-machine-learning/>

Gu, J., & Xu, H. (2022, January). An ensemble method for phishing websites detection based on xgboost. In 2022 14th international conference on computer research and development (ICCRD) (pp. 214-219). IEEE.

REDDY, K. T. (2024) Enhancing Phishing Detection Through Multilayer Perceptron in Cybersecurity.

Shahrivari, V., Darabi, M. M., & Izadi, M. (2020). Phishing detection using machine learning techniques. arXiv preprint arXiv:2009.11116.

Singh, P., & Maurya, H. ML TECHNIQUES FOR PHISHING DETECTION.

Dataset:

<https://www.kaggle.com/datasets/naserabdullahalam/phishing-email-dataset/data>